The Art of Readable Code

Book Review — Dustin Boswell & Trevor Foucher (O'Reilly, 2011)

HEM Bellyday

How the Book Is Organized

Chapter 1 — Code Should Be Easy to Understand (the main idea)

Part I — Surface-Level Improvements
Ch. 2 Packing Information into Names Ch. 3 Names That Can't Be Misconstrued
Ch. 4 Aesthetics Ch. 5 Knowing What to Comment Ch. 6 Making Comments Precise and Compact

Part II — Simplifying Loops and Logic
Ch. 7 Making Control Flow Easy to Read Ch. 8 Breaking Down Giant Expressions
Ch. 9 Variables and Readability

Part III — Reorganizing Your Code
Ch. 10 Extracting Unrelated Subproblems Ch. 11 One Task at a Time
Ch. 12 Turning Thoughts into Code Ch. 13 Writing Less Code

Part IV — Selected Topics
Ch. 14 Testing and Readability Ch. 15 Designing a "Minute/Hour Counter"

Chapter 1 — Theory and Definitions

Code Should Be Easy to Understand

- **Readability** — how quickly and correctly a person can understand a piece of code.
- **Time-till-understanding** — the time it takes someone else to understand your code. This is the book's main measure of code quality.
- **Understanding** — not just reading the code, but being able to modify it, find its bugs, and know how it interacts with the rest of the program.
- **The Fundamental Theorem of Readability** — code should be written to minimize the time it would take for someone else to understand it.
- **Conflict with other goals?** — the authors argue there is usually none: readable code also tends to be well designed, efficient, and easy to test.

Chapter 1 — Code Should Be Easy to Understand

The Fundamental Theorem of Readability:

Code should be written to minimize the time it would take for someone else to understand it.

- "Someone else" includes *you* six months later.
- Understanding = being able to change the code and spot its bugs.
- Fewer lines is not automatically better: short but cryptic code costs more time.
- Readable code usually also has better design and is easier to test.

Chapter 1 — Example: Shorter Is Not Always Better

Compact but hard to read:

```
return exponent >= 0 ? mantissa * (1 << exponent)
                     : mantissa / (1 << -exponent);
```

Longer but faster to understand:

```
if (exponent >= 0) {
    return mantissa * (1 << exponent);
} else {
    return mantissa / (1 << -exponent);
}
```

Lesson: measure code by *time-till-understanding*, not by line count.

Part I — Surface-Level Improvements

- Small, low-risk changes made one line at a time.
- No need to restructure the program.
- Three areas: **naming**, **layout (aesthetics)**, and **comments**.

Chapter 2 — Theory and Definitions

Packing Information into Names

- **Name as a tiny comment** — a name is read every time the variable or function is used, so it should carry as much meaning as a short comment.
- **Specific word** — a word that says exactly what happens (*Fetch*, *Download*) instead of a vague one (*Get*).
- **Generic name** — an empty name like `tmp`, `retval`, `foo` that says nothing about the value's purpose.
- **Concrete vs. abstract name** — a concrete name describes *what the code actually does*; an abstract name describes a vague idea.
- **Attached information** — extra details placed inside a name: units (`_ms`), state (`raw_`, `unsafe_`), encoding (`_utf8`).
- **Scope** — the region of code where a name is visible. Larger scope needs a longer, clearer name.
- **Naming convention** — consistent use of capitalization and underscores so the *form* of a name shows its kind (class, constant, member).

Chapter 2 — Packing Information into Names (1)

Key idea: a name is a tiny comment. Pack information into it.

Choose specific words

- `GetPage(url)` — from cache? from database? from internet?
- Better: `FetchPage(url)` or `DownloadPage(url)`
- `Size()` → `Height()`, `NumNodes()`, `MemoryBytes()`

Avoid generic names like `tmp`, `retval`, `i`, `j`

```
retval += v[i] * v[i];      // what is retval?
sum_squares += v[i] * v[i]; // purpose is clear
```

Exception: `tmp` is fine for a short-lived swap.

Chapter 2 — Packing Information into Names (2)

Prefer concrete names over abstract names

- **Bad:** `ServerCanStart()` → **Better:** `CanListenOnPort()`

Attach extra information (units, state)

- **Bad:** `start` → **Better:** `start_ms` **Bad:** `size` → **Better:** `size_mb`
- **Bad:** `password` → **Better:** `plaintext_password` **Bad:** `html` → **Better:** `html_utf8`

Name length should match scope

- Small scope (a few lines): short name is OK, e.g. `m`.
- Large scope (many screens): use a longer, descriptive name.
- Drop useless words: `ConvertToString()` → `ToString()`.

Use formatting to convey meaning

- `CamelCase` for classes, `lower_case` for variables, `kConstantName`, `member_` with underscore.

Chapter 3 — Theory and Definitions

Names That Can't Be Misconstrued

- **Misconstrued name** — a name that a reader can reasonably interpret in a different way from what the writer meant.
- **Devil's advocate test** — before choosing a name, actively look for the *wrong* meanings someone could read from it.
- **Inclusive limit** — a boundary value that is itself allowed (`max_`, `min_`).
- **Inclusive range** — both ends are included: [first, last].
- **Inclusive/exclusive range** — start included, end excluded: [begin, end).
- **Boolean name** — a name for a true/false value; it should read like a yes/no question (`is_`, `has_`, `can_`) and avoid negation.
- **User expectation** — the meaning programmers already associate with common words (e.g. `get` and `size` are assumed to be cheap).

Chapter 3 — Names That Can't Be Misconstrued (1)

Key idea: ask yourself, *"What other meanings could someone read from this name?"*

`filter()` **is ambiguous**

- `results = Database.all_objects.filter("year <= 2011")`
- Does it *keep* or *remove* those rows?
- Better: `select()` or `exclude()`

`Clip(text, length)` **is ambiguous**

- Remove `length` from the end? Or cut down to at most `length`?
- Better: `Truncate(text, max_chars)`

Chapter 3 — Names That Can't Be Misconstrued (2)

Limits and ranges

- Inclusive limit: use `max_` / `min_` e.g. `MAX_ITEMS_IN_CART`
- Inclusive range: use `first` / `last`
- Inclusive/exclusive range: use `begin` / `end`

Naming booleans

- Use `is`, `has`, `can`, `should`: `need_password`, `user_is_authenticated`
- Avoid negations: **Bad:** `disable_ssl` → **Better:** `use_ssl`

Match user expectations

- People expect `get()` and `size()` to be cheap.
- An expensive `getMean()` should be renamed `computeMean()`.

Chapter 4 — Theory and Definitions

Aesthetics

- **Aesthetics** — the visual layout of code: spacing, line breaks, alignment, and ordering. Like the layout of a magazine, it makes code easy to scan.
- **Consistent layout** — using the same patterns everywhere so the reader gets used to them and reads faster.
- **Similar code looks similar** — code that does similar things should have the same shape, so differences stand out.
- **Column alignment** — lining up related values vertically so the reader can compare them at a glance.
- **Code paragraph** — a group of related lines separated from others by a blank line, often with a short summary comment.
- **Aesthetics vs. design** — two independent ideas: aesthetics is the surface look; design is the structure (functions, classes). Good code needs both.

Chapter 4 — Aesthetics (1)

Three principles

- Use a consistent layout the reader can get used to.
- Make similar code look similar.
- Group related lines into blocks.

Column alignment when it helps

```
CheckFullName("Doug Adams"  , "Mr. Douglas Adams" , "");
CheckFullName(" Jake  Brown ", "Mr. Jake Brown III", "");
CheckFullName("No Such Guy" , ""                   , "no match found");
```

Now the differences between lines are easy to see.

Chapter 4 — Aesthetics (2)

Use methods to clean up irregularity

- Repeated, messy test lines → one helper function per line.

Pick a meaningful order and keep it

- If HTML form fields are `first_name`, `last_name`, `email`, use the same order in code.
- Or order from most important to least important.

Organize declarations and code into "paragraphs"

- Group related methods (handlers, query helpers, ...).
- Separate logical steps with blank lines, like paragraphs in an essay.

Consistency over personal style

- Brace placement does not matter much. Being consistent across the project does.

Chapter 5 — Theory and Definitions

Knowing What to Comment

- **Purpose of a comment** — to help the reader know as much as the writer knew when writing the code.
- **Low-value comment** — a comment that only repeats what the code already says clearly.
- **Crutch comment** — a comment that tries to make up for bad code (e.g. a bad name). Good code $>$ bad code $+$ good comment.
- **Director commentary** — a comment that records your insight: *why* the code is this way, what you tried, what is surprising.
- **Comment markers** — standard tags for known problems: `TODO` (not done yet), `FIXME` (known bug), `HACK` (inelegant solution), `XXX` (danger, major problem).
- **Big-picture comment** — a high-level comment at the file or class level explaining how parts fit together.
- **Summary comment** — a short comment on top of a block of code that tells what the block does.

Chapter 5 — Knowing What to Comment (1)

Key idea: a comment should help the reader know what the writer knew.

What NOT to comment

- Facts that can be read quickly from the code itself.
- "Crutch comments" that excuse bad names — fix the name instead.

```
// The class definition for Account     <- useless
class Account { ... };

// Release the handle for this key (does not modify registry)
void DeleteRegistry(RegistryKey* key);  <- crutch comment
void ReleaseRegistryHandle(RegistryKey* key);  <- just rename
```

Chapter 5 — Knowing What to Comment (2)

What you SHOULD record

- "Director commentary": why the code is this way and not another.
- Known flaws, with markers: `TODO:`, `FIXME:`, `HACK:`, `XXX:`
- The story behind a constant:

```
NUM_THREADS = 8  # as long as it's >= 2 * num_processors, that's good enough.
```

Put yourself in the reader's shoes

- Answer the questions a reader will ask ("Huh? why?").
- Warn about surprising behavior and pitfalls.
- Write big-picture comments at the file/class level.
- Summarize blocks of code so the reader does not get lost in details.

Chapter 6 — Theory and Definitions

Making Comments Precise and Compact

- **Precise comment** — a comment that is as specific and detailed as possible, leaving no room for a wrong reading.
- **Compact comment** — a comment that uses as little space as possible, because comments take screen space and reading time.
- **Information-to-space ratio** — how much useful information a comment carries per line. The goal is a high ratio.
- **Ambiguous pronoun** — a word like *it*, *this*, *that* which could refer to more than one thing.
- **Input/output example** — a carefully chosen example in a comment that shows exactly how a function behaves, especially at corner cases.
- **Intent** — the high-level goal of the code (what it is *for*), as opposed to the literal steps it performs.
- **Information-dense word** — a term that packs a whole idea into one word: *cache*, *canonicalize*, *heuristic*, *brute-force*.

Chapter 6 — Making Comments Precise and Compact

Key idea: comments should have a high *information-to-space ratio*.

- Avoid ambiguous pronouns ("it", "this").
- Polish sloppy sentences; describe function behavior precisely.
- Use input/output examples that show corner cases:

```
// Example: Strip("abba/a/ba", "ab") returns "/a/"
```

- State the *intent*, not the obvious steps:

```
// Display each price, from highest to lowest
for (list<Product>::reverse_iterator it = products.rbegin(); ...)
```

- "Named parameter" comments: `Connect(/* timeout_ms = */ 10, /* use_encryption = */ false);`
- Use dense words: "caches", "canonicalize", "heuristic", "brute-force".

Part II — Simplifying Loops and Logic

- Goes deeper: control flow, logical expressions, and variables.
- Goal: reduce the reader's **"mental baggage"**.
- Complicated loops, giant expressions, and many variables make the reader remember more, so bugs hide and changes become hard.

Chapter 7 — Theory and Definitions

Making Control Flow Easy to Read

- **Control flow** — the order in which statements execute: conditionals, loops, jumps, returns. This is where code becomes hardest to follow.
- **Changing vs. stable value** — in a comparison, the value being examined (changing) goes on the left; the value compared against (more constant) goes on the right.
- **"Yoda notation"** — writing `if (NULL == obj)`; once used to avoid bugs, now unnecessary since compilers warn about it.
- **Ternary operator** `?:` — a compact if/else expression. Use only when it reduces time-till-understanding.
- **Early return (guard clause)** — leaving a function as soon as a special case is handled, instead of wrapping everything in an `else`.
- **Nesting** — placing blocks inside other blocks. Each level is another condition the reader must keep in mind.
- **Hidden flow constructs** — threads, signal/interrupt handlers, exceptions, function pointers, virtual methods: they make execution order hard to trace.

Chapter 7 — Making Control Flow Easy to Read (1)

Order of arguments in conditionals

- Left side: the value being questioned (changing). Right side: the value compared against (stable).

```
if (length >= 10)   // reads naturally
if (10 <= length)   // harder to read
```

Order of if/else blocks

- Prefer the positive case first: `if (debug)` not `if (!debug)`.
- Prefer the simpler case first.
- Prefer the more interesting or important case first.

Ternary `?:` only for simple cases

```
time_str += (hour >= 12) ? "pm" : "am";   // fine
```

Chapter 7 — Making Control Flow Easy to Read (2)

- **Avoid** `do/while`: the condition is far below the code it controls.
- **Return early** (guard clauses) to handle special cases first.
- **Avoid** `goto` except the simple "exit/cleanup" case.
- **Minimize nesting**: each level is one more condition to remember.

```
for (int i = 0; i < results.size(); i++) {
    if (results[i] == NULL) continue;
    non_null_count++;
    if (results[i]->name == "") continue;
    cout << "Considering candidate..." << endl;
}
```

- Be careful with threads, signal handlers, exceptions, callbacks: they make the flow hard to follow.

Chapter 8 — Theory and Definitions

Breaking Down Giant Expressions

- **Giant expression** — an expression too long to understand in one read. People can only think about about 3–4 things at once.
- **Explaining variable** — an extra variable that holds a sub-expression and whose name explains what it means.
- **Summary variable** — a variable that replaces a larger expression which is easy to understand but used often, so it is easier to manage.
- **De Morgan's Laws** — rules for rewriting negated boolean expressions:

$$\neg(a \vee b \vee c) \equiv \neg a \wedge \neg b \wedge \neg c \qquad \neg(a \wedge b \wedge c) \equiv \neg a \vee \neg b \vee \neg c$$

- **Short-circuit evaluation** — `&&` and `||` stop as soon as the result is known. Using this as hidden if-statement logic is "clever" but hard to read.
- **DRY (Don't Repeat Yourself)** — pulling repeated sub-expressions into one place reduces typos and makes differences visible.

Chapter 8 — Breaking Down Giant Expressions (1)

Key idea: break huge expressions into smaller, named pieces.

Explaining variable

```
if line.split(':')[0].strip() == "root": ...

username = line.split(':')[0].strip()
if username == "root": ...
```

Summary variable

```
final boolean user_owns_document = (request.user.id == document.owner_id);
if (!user_owns_document) { /* document is read-only */ }
```

De Morgan's Laws

```
if (!(file_exists && !is_protected)) Error("Sorry...");
if (!file_exists || is_protected)    Error("Sorry...");
```

Chapter 8 — Breaking Down Giant Expressions (2)

Don't abuse short-circuit logic

```
assert((!(bucket = FindBucket(key))) || !bucket->IsOccupied());

bucket = FindBucket(key);
if (bucket != NULL) assert(!bucket->IsOccupied());
```

Wrestling with complicated logic

- Example: `Range::OverlapsWith()` — instead of checking every overlap case, check the opposite ("no overlap") which is much simpler.

Break down giant statements

- Pull repeated sub-expressions into variables at the top of the function (DRY).
- Bonus: macros or helpers can simplify long repetitive code.

Chapter 9 — Theory and Definitions

Variables and Readability

- **Mental baggage** — the amount of information a reader must hold in mind; every extra variable adds to it.
- **Three variable problems** — (1) more variables are harder to track, (2) bigger scope means tracking them longer, (3) more changes make the current value harder to know.
- **Useless temporary variable** — a variable that does not break down a complex expression, add clarity, or avoid duplication.
- **Control flow variable** — a variable (like `done`) whose only job is to steer a loop or function; often replaceable by `break` or `return`.
- **Scope shrinking** — making a variable visible to as few lines as possible ("limit its exposure").
- **Write-once variable** — a variable assigned only once (`const`, `final`, immutable). The fewer places a variable changes, the easier it is to reason about.

Chapter 9 — Variables and Readability

Three problems: more variables, bigger scope, more changes ⇒ harder to track.

1. **Eliminate useless variables**

```
now = datetime.datetime.now()
root_message.last_view_time = now      # 'now' adds nothing

root_message.last_view_time = datetime.datetime.now()
```

Also remove unnecessary control-flow variables like `done = true`; use `break`/`return`.

2. **Shrink the scope**
   - Avoid globals; make class members into locals when possible.
   - Define variables close to their first use.
3. **Prefer write-once variables**
   - Use `const` / `final`. The fewer places a variable changes, the easier it is to reason about.

Part III — Reorganizing Your Code

- Bigger changes at the function level.
- Three ideas:
  - Extract "unrelated subproblems" (Ch. 10)
  - Do one task at a time (Ch. 11)
  - Describe the code in plain words first (Ch. 12)
- Plus: the best code is the code you never write (Ch. 13).

Chapter 10 — Theory and Definitions

Extracting Unrelated Subproblems

- **Engineering** — breaking big problems into smaller ones and putting the solutions back together.
- **High-level goal** — the main purpose of a function or block of code.
- **Unrelated subproblem** — a part of the code that is needed but does not directly work toward the high-level goal (e.g. distance math inside a "find closest" function).
- **Utility code** — basic general-purpose code (string handling, file reading, hashing) that is independent of the project.
- **General-purpose code** — code decoupled from the rest of the project; easier to understand, test, and reuse.
- **Interface** — the way a piece of code is called (its inputs and outputs). A clean interface hides messy details.
- **Wrapper function** — a small function that reshapes an awkward interface into one that fits your needs.

Chapter 10 — Extracting Unrelated Subproblems

Method

1. What is the high-level goal of this function?
2. For each line, ask: does it work toward that goal directly, or solve an unrelated subproblem?
3. If enough lines solve an unrelated subproblem, move them to a separate function.

Example: `findClosestLocation()`

```
// before: the loop body is full of sin/cos/acos math
// after:
var dist = spherical_distance(lat, lng, array[i].latitude,
                                        array[i].longitude);
```

- Build a lot of general-purpose utility code (`ReadFileToString()`, `format_pretty()`).
- Simplify or reshape bad interfaces with wrapper functions.
- But don't go too far: dozens of tiny functions also hurt readability.

Chapter 11 — Theory and Definitions

One Task at a Time

- **Task** — one distinct thing the code is doing. A task can be very small (e.g. "make sure the object is valid").
- **Mixed-task code** — code that does many tasks at once, interleaved; hard to understand because the reader must track all tasks together.
- **One task at a time** — the principle that code should be organized so each section or function does only one task.
- **Defragmenting code** — reordering and grouping code so all lines for the same task sit together.
- **Relation to Ch. 10** — Ch. 10 extracts subproblems into new functions; Ch. 11 separates tasks, which may or may not become new functions.

Chapter 11 — One Task at a Time

Key idea: code should be organized to do only *one task at a time*.

Method

1. List all the tasks the code is doing ("task" can be very small).
2. Separate those tasks into different functions, or at least different sections.

Example: voting widget (up/down/none)

- Messy version mixes two tasks: parsing the old/new vote and updating the score.
- Clean version: `vote_value()` converts a vote to a number, then `score -= vote_value(old); score += vote_value(new);`

Larger example: extracting fields from a location object — solve each task (pick city, pick country, handle fallback) separately.

Chapter 12 — Theory and Definitions

Turning Thoughts into Code

- **Main idea** — if you can't explain something simply, you don't understand it well enough (quote attributed to Einstein).
- **Plain English description** — describing what the code must do in simple words, as if to a colleague, before or while writing the code.
- **Rubber ducking** — explaining a problem out loud (even to an object such as a rubber duck); often the act of explaining reveals the solution.
- **Key words** — the important words and phrases in the description; they guide the structure of the code.
- **Knowing your libraries** — a large part of writing compact code is knowing what existing functions can do.
- **Top-down application** — the same method works for big problems: describe the steps, then turn each step into a function.

Chapter 12 — Turning Thoughts into Code

Key idea: if you can't explain it simply, you don't understand it well enough (Einstein quote).

Method

1. Describe what the code must do in plain English, like explaining to a colleague ("rubber ducking").
2. Notice the key words and phrases.
3. Write code that matches that description.

Example: permission check in PHP

```
// "Two ways you can be authorized:
//  1) you are an admin, or
//  2) you own the current document (if there is one).
//  Otherwise, you are not authorized."
if (is_admin_request()) {
    // authorized
} elseif ($document && ($document['username'] == $_SESSION['username'])) {
    // authorized
} else {
    return not_authorized();
}
```

Knowing your libraries helps: the right function often turns a messy loop into one line.

Chapter 13 — Theory and Definitions

Writing Less Code

- **Main idea** — the most readable code is no code at all. Every line must be tested, documented, and maintained.
- **YAGNI (You Aren't Gonna Need It)** — don't implement features before they are actually needed; programmers tend to overestimate what is essential.
- **Requirement** — what the program must do. Questioning requirements often reveals a simpler problem that solves most real cases.
- **Codebase size** — the total amount of code; as it grows, it becomes harder to understand and change. Keep it small and lightweight.
- **Dead code** — unused code; it should be deleted (version control keeps the history).
- **Library familiarity** — knowing the standard library and tools so you can reuse instead of rewriting.

Chapter 13 — Writing Less Code

Key idea: the most readable code is *no code at all*.

- **You aren't gonna need it**: don't implement features nobody needs yet.
- **Question and break down requirements**: often a simpler version solves 90% of the problem (e.g. a store locator for one state does not need to handle the poles or the date line).
- **Keep the codebase small**: delete unused code, reuse utilities, keep projects as separate subprojects.
- **Know your libraries**: spend 15 minutes every few months reading the standard library's functions.
- **Use Unix tools instead of coding**: a one-line `grep | sort | uniq` can replace a whole script.

Part IV — Selected Topics

- Apply the ideas of the whole book to two practical areas:
  - Writing readable **tests** (Ch. 14)
  - A full design walkthrough: the **minute/hour counter** (Ch. 15)

Chapter 14 — Theory and Definitions

Testing and Readability

- **Test code** — code that checks the behavior of other code. It is real code and deserves the same readability.
- **Readable test** — a test that hides unimportant details so the essential inputs and expected outputs are obvious.
- **Test helper / mini-language** — small functions or compact notations that let each test be written in one clear line.
- **Readable error message** — a failure message that shows input, expected output, and actual output.
- **Good test input** — the simplest set of inputs that still fully exercises the code (including corner cases).
- **Test naming** — the name should describe the function and situation tested: `Test_<Function>_<Situation>`.
- **Test-friendly development** — designing code to be easy to test; signs of hard-to-test code: global variables, many external dependencies, non-deterministic behavior.

Chapter 14 — Testing and Readability (1)

Key idea: test code should be just as readable as real code.

- Hide less important details; show only what matters for the test.
- Write tests so each one reads like a short sentence.

```
// Before: 20 lines of building ScoredDocument objects...
// After:
CheckScoresBeforeAfter("2, 1, 3", "3, 2, 1");
CheckScoresBeforeAfter("0, -0.1, -10", "0");
```

- Make error messages readable: print the input, the expected, and the actual output.

Chapter 14 — Testing and Readability (2)

Choosing good test inputs

- Prefer clean, simple inputs that still fully exercise the code.
- Use several small tests instead of one giant "kitchen sink" test.

Naming test functions

- Pattern: `Test_<FunctionName>_<Situation>()`
- e.g. `Test_SortAndFilterDocs_NegativeValues()`

Test-friendly development

- Code that is easy to test usually has a well-defined interface, little global state, few hidden dependencies.

Going too far

- Don't sacrifice readability of real code just for tests; don't chase 100% coverage at any cost.

Chapter 15 — Theory and Definitions

Designing a Minute/Hour Counter

- **Purpose of the chapter** — a full case study that applies all earlier techniques (naming, comments, one task at a time, simple design) to one real problem.
- **Problem definition** — record the number of bytes a server sends and report totals for the last 60 seconds and last 3600 seconds.
- **Class interface** — the public methods of a class; clear names and precise comments are defined before implementation.
- **Design trade-offs** — comparing solutions on readability, speed (time per call), memory use, and precision.
- **Conveyor belt (queue)** — a structure where new events enter at one end and old events fall off the other.
- **Time bucket** — a container that sums all events in a fixed time slot, giving fixed memory at the cost of small precision loss.

Chapter 15 — Minute/Hour Counter (1)

Problem: track how many bytes a web server transferred in the last minute and last hour.

Defining the class interface

- Rename `Count()` → `Add(count)`; clarify `MinuteCount()` / `HourCount()`.
- Rewrite vague comments to be precise ("in the past 60 seconds").

Attempt 1: Naive solution

- Store every event in a list; count by walking the list.
- Hard to read, slow, and memory grows forever.

Attempt 2: Conveyor belt design

- Two queues (minute, hour); old events fall off the end. Faster, but still stores every event.

Chapter 15 — Minute/Hour Counter (2)

Attempt 3: Time-bucketed design

- Group events into time buckets (e.g. 60 buckets of 1 second, 60 buckets of 1 minute).
- Small, fixed memory; fast; slight loss of precision is acceptable.
- Implemented with small classes: `ConveyorQueue` and `TrailingBucketCounter`.

Comparing the three solutions

- Naive: few lines but bad performance.
- Conveyor belt: correct but unbounded memory.
- Time-bucketed: more lines in total, but each class is simple and does one task.

Lesson: splitting into small, focused classes made a hard problem readable.

Summary — What We Learn

- **One rule**: minimize the time for someone else to understand your code.
- **Names**: specific, concrete, with units, impossible to misread.
- **Layout**: consistent, aligned, grouped into paragraphs.
- **Comments**: explain *why*, flaws, and surprises — not the obvious.
- **Logic**: simple conditions, early returns, little nesting, small expressions.
- **Variables**: fewer, smaller scope, write-once.
- **Structure**: extract subproblems, one task at a time, describe in plain words first.
- **Less code**: don't build what you don't need; reuse libraries.
- **Tests**: readable tests and readable error messages.

Personal Reflection

- Most advice is small and practical — it can be used from tomorrow.
- Most useful for me: units in names (`_ms`, `_mb`), early return, explaining variables.
- Relevant for data and modeling code: notebooks and scripts are read many more times than they are written.
- Weak point: examples are mostly C++, Java, Python, JavaScript from 2011; some style points are now handled by formatters and linters.

Thank you